Presenter Notes: Proof of Repair

Final Review, 28-10-2026 · Your parts: Experiments & Results · Conclusion & Future Work · Demo · Limitations

0. The whole system in 60 seconds (memorise this)

> *"A buyer of a second-hand car, phone or appliance gets shown a repair history, but invoices and PDFs can be edited or back-dated. Our system makes that history tamper-evident. A workshop enters the device ID (VIN / IMEI), the fault code and symptoms. A LoRA fine-tuned LLM writes a structured JSON repair report. We hash that report and bind it to the device ID, odometer/cycle count, evidence and model version in one 'proof leaf'. Many leaves are combined into a Merkle tree, and only the 32-byte Merkle root goes on the blockchain through our smart contract, plus a Bitcoin timestamp through OpenTimestamps. Later anyone types the VIN and we recompute every hash. If even one character in the database changed, the hashes no longer match the root on-chain, and the record shows TAMPERED."*

```
DTC / symptoms  →  LoRA LLM (structured JSON)  →  SHA-256 report hash  →  PoR leaf (VIN + odometer + report hash + evidence CID + model)
→  Merkle tree over a batch  →  root anchored in ProofOfRepair.sol (Ethereum)  +  OpenTimestamps (Bitcoin)
→  Buyer enters VIN  →  verify_full(): report_hash · leaf_digest · evidence · merkle · chain_anchor  →  VERIFIED / TAMPERED
```

How to explain each mechanism simply

Concept	One-line explanation
SHA-256 hash	A fingerprint of the data. Change one character and the fingerprint changes completely. We serialise the JSON with sorted keys (*canonical_dumps*) so the same report always gives the same hash.
Evidence CID	Photos/files stay off-chain; we store only their content hash, so evidence is addressed by what it contains, not by a filename.
PoR leaf	Binds *who/what* together: VIN, odometer, report hash, evidence CID and model provenance. You cannot move a report to another car, or roll back the odometer, without changing the leaf.
Merkle tree	Pairs of hashes are hashed upward until one root remains. One root covers a whole batch of repairs, so one transaction serves many records (cheap). A proof for one record needs only $\log_2(n)$ sibling hashes. Leaves are prefixed 0x00 and inner nodes 0x01 (domain separation, so an inner node cannot be passed off as a leaf).
Smart contract anchor	*ProofOfRepair.sol → anchorMerkleRoot(root)* stores the root with a block timestamp. The blockchain is append-only, so nobody, including us, can change it later.
OpenTimestamps	A second, independent proof: the root is committed into Bitcoin via public calendar servers. Status goes pending → confirmed in a few hours.
Verification	Recompute report hash → recompute leaf → walk the Merkle path → check the root exists on-chain. All must pass for VERIFIED. Runs in well under a second.
Off-chain vs on-chain	Data (reports, evidence, registry) lives in PostgreSQL. Only the root + anchor metadata goes public. Privacy-friendly and cheap.

What changed since the slides were made (mention if asked): the project now covers **phones, cars and appliances** (multi-domain LoRA), anchoring now runs on the **public Ethereum Sepolia testnet** (contract 0x37C9…b6d9, viewable on Etherscan), the registry runs on **PostgreSQL 15** (SQLite is only a

fallback; slide 9 still says SQLite), and the LLM is **grounded** on a knowledge base of **4,666 real repair procedures** (iFixit/MyFixit + MechanicDB) via full-text search. The UI shows which guide each report was "Grounded on".

1. Experiments and Results (slide 12)

What we did (setup)

- **Task:** given a fault code + symptoms, output a 9-field JSON report (fault code, diagnosis, severity, parts, steps, QA checks, minutes, confidence, buyer advice).
- **Models:** TinyLlama-1.1B with LoRA (3 epochs) vs Mistral-7B-Instruct, 4-bit, LoRA trained with Apple MLX (2 epochs). LoRA trains only small low-rank matrices, so a 7B model fine-tunes on a laptop.
- **Test data:** a **held-out** set of **57 real OBD-II cases**. "Held-out" = never seen during training.

What each metric means

Metric	Meaning	Why it matters
Fault-code accuracy	Predicted code exactly equals the true DTC	Wrong code = wrong repair record
JSON valid	Output parses as JSON	Invalid output is **rejected before sealing**; we never hash garbage
Schema compliance	All 9 required keys present	Verifier and UI rely on fixed fields
Severity accuracy	Predicted low/medium/high matches label	Subjective; labels are noisy
ROUGE-L	Word-overlap of free text with reference	Low is expected: many valid ways to phrase a fix

Results (read the table like this)

Model	Fault code	JSON valid	Schema	Severity	ROUGE-L
TinyLlama-1.1B LoRA	93.0% (53/57)	**100%**	**100%**	56.1%	0.133
Mistral-7B 4-bit LoRA	**98.3% (56/57)**	98.3%	98.3%	59.7%	0.239

> "*The bigger model, Mistral, is more accurate on the fault code: 56 of 57. The small model, TinyLlama, never broke the JSON format, 57 of 57. That trade-off matters to us because a malformed report is rejected before it can be sealed, so format reliability is a safety property, not just a quality score. Severity is around 56–60% because severity labels are subjective. ROUGE is low because there are many correct ways to word repair steps. With only 57 cases, we present these as indicative, not final.*"

Extra result: multi-domain model (new work, not on the slide)

- Dataset: 4,666 repair procedures (3,682 phone, 609 appliance, 375 car) from iFixit/MyFixit, MechanicDB OBD-II and NHTSA complaints. Split: **1,625 train / 70 valid / 70 test**, seed 42. The dataset itself has a Merkle root, so the training data is also fingerprinted.
- Mistral-7B + multi-domain LoRA, evaluated on 30 test examples (10 per domain), ~13 s per example:

Domain	n	JSON valid	Fault code exact	Severity
Appliance	10	100%	100%	80%
Car	10	100%	90%	50%

Phone	10	90%	70%	60%
Overall	30	**96.7%**	**86.7%**	63.3%

> *"When we extended from cars to phones and appliances, one model stayed at 96.7% valid JSON and 86.7% exact fault codes overall. Phones are hardest because many phone faults share symptoms. Again, 30 cases is small, so this shows feasibility, not a benchmark."*

2. Conclusion and Future Work (slide 13)

> *"We built a working prototype that connects AI-assisted diagnosis with verifiable repair provenance. Diagnosis and record integrity are usually solved separately; we bind them into one VIN-specific attestation that a buyer can check in one step."*

Achieved: say each in one breath

- **VIN-bound PoR:** every report is tied to a specific vehicle/device and odometer.
- **LoRA diagnosis:** structured JSON output, 93–98% fault-code accuracy on real OBD data.
- **Evidence hashing:** evidence off-chain, content IDs in the leaf.
- **Merkle batching:** one transaction anchors many repairs; per-record membership proofs.
- **EVM anchoring + OpenTimestamps:** two independent public time proofs (Ethereum + Bitcoin).
- **Buyer UI:** type VIN/IMEI, see timeline + VERIFIED/TAMPERED.
- **Evaluation:** documented on a real held-out OBD set, plus the multi-domain extension.

Future work: why each one matters

Item	What and why
Workshop review & approval	A mechanic must check and correct the AI report **before** sealing; sealing a wrong report makes a wrong record permanent.
Authentication & signatures	Today the system proves a record *was not changed*; signatures would prove *which workshop* created it.
Live blockchain anchoring	Contract is already live on Sepolia; next is mainnet or a low-cost Layer-2, per-workshop wallets, and automatic OTS receipt upgrade/verification.
Cloud deployment	Managed DB, object storage for evidence, secured API.
Larger evaluation	Hundreds/thousands of real cases per domain; human-rated quality of repair steps.
Physical OBD-II + QR	Read codes directly from an OBD dongle; QR sticker on the device for one-scan buyer lookup.

3. Demo script (≈8–10 min) on the public Ethereum Sepolia testnet

Our deployment (have these ready)

Item	Value
Network	**Ethereum Sepolia** public testnet, chain id **11155111**, ~12 s per block
Smart contract (ProofOfRepair)	**0x37C9383cdCBb52126D238925a6EDbcccF8b3b6d9** sepolia.etherscan.io/address/0x37C9383cdCBb52126D238925a6EDbcccF8b3b6d9
Workshop wallet (deployer + submitter)	0x133809b90a48303b6C7A51aB8dEb7C4b7e0aF609 (≈0.041 SepoliaETH on 28-10-2026)

Deployed	28-10-2026, deploy tx 0xdc6a88b9…a74b, block 12,010,296
Source code	**Verified on Etherscan** (ProofOfRepair, Solidity 0.8.20): Contract tab has a green ✓, transactions show the decoded method name, and anyone can use **Read Contract**.
Anchors so far (28-10)	3 batches: blocks 12,010,314 (tx 0x11b3…, 3 records), 12,010,357 (0x16b1…), 12,010,431 (0xf1eb…). Plus 1 reverted attempt (0x999a…, see Q&A).
RPC endpoint the server talks to	https://ethereum-sepolia-rpc.publicnode.com

Before the panel arrives

One click: double-click **START_DEMO.command** in the project folder (also after opening the laptop lid; it restarts whatever stopped). To stop: **STOP_DEMO.command** (PostgreSQL keeps running).

```
[1/5] PostgreSQL              OK  database proof_of_repair is up
[2/5] Blockchain network      OK  network: sepolia      (falls back to local Hardhat if no
internet)
[3/5] Proof of Repair server  OK  running on port 8080  (opens the server window if
needed)
[4/5] Database <-> blockchain audit   RESULT: DATABASE AND BLOCKCHAIN ARE CONSISTENT
      Bitcoin timestamps (OpenTimestamps): CONFIRMED / pending per batch
      files for opentimestamps.org: artifacts/ots_public/
[5/5] Opening   web app + contract on sepolia.etherscan.io + pgAdmin
```

Manual alternative: *bash scripts/start_demo.sh sepolia*. Optional clean history: *python scripts/init_db.py --reset* (keeps the 4,666 guides, but then the audit has nothing to show).

- No Hardhat window opens: the blockchain is the **public Sepolia network**. Your "blockchain window" is now **Etherscan** (START opens the contract page for you).
- **10 min before:** run *.venv/bin/python scripts/ots_status.py --db-only*. If a batch says **CONFIRMED … Bitcoin block N**, you can show the mempool.space link live (Bitcoin confirmation takes a few hours after anchoring).
- Warm the model: seal one case first (Mistral loads in ~20 s). Check the top bar says **LIVE · Ethereum Sepolia · id 11155111 · block …**.
- Open pgAdmin → Query Tool → load *docs/demo_queries.sql*.

#	Do	Say
1	Point at top bar: PostgreSQL · N guides, **LIVE · Ethereum Sepolia · id 11155111 · block N**, OTS live	"Three real back ends: our Postgres database, the **public** Ethereum Sepolia test network (that block number is live and keeps rising) and OpenTimestamps on Bitcoin."
2	Phone tab → **iPhone 6 · battery** → Seal repair	"It searched 4,666 real repair manuals, found the matching iFixit guide (Grounded on card), and Mistral+LoRA wrote the structured report. Here is the report hash and the leaf digest, which binds IMEI, battery cycles, report, evidence and model." (No blockchain yet: sealing is local and free.)
3	Car tab → **Civic · P0420**; Appliance tab → **Kenmore washer · drain**	"Same pipeline for a car OBD-II code and an appliance fault."
4	pgAdmin: queries 1–5; show *v_device_history*	"Records are in normal tables. They are sealed but not yet anchored."

5	Section 03 → **Anchor pending batch**. Wait ~15–30 s.	"Three records → one Merkle root → one transaction on the public Ethereum network. We wait for a real Sepolia block to include it."
6	**Click the tx hash** in the UI → Etherscan opens	"Anyone in the world can open this page. Status Success, the block number, the timestamp, from our workshop wallet to our ProofOfRepair contract." (See 3c for every field.)
7	Etherscan → Logs tab, then footer link **Smart contract** → contract page	"The contract emitted MerkleRootAnchored with our root. The contract page lists every batch we have ever anchored."
7b	Contract page → **Contract ✓** tab → **Read Contract** → *isRootAnchored* → paste the Merkle root (prefix 0x) → Query	"Anyone can ask the contract directly, without our server: it answers **true**. And the source code is verified, so you can read exactly what the contract does."
8	pgAdmin query 6	"The database stores the same tx hash, block number and Etherscan link."
8b	Terminal: *.venv/bin/python scripts/check_db_chain.py*	"An automatic audit: for every batch it checks the receipt, the block, the event and isRootAnchored on Sepolia; for every record it recomputes the hashes and the Merkle path. Result: **DATABASE AND BLOCKCHAIN ARE CONSISTENT**."
8c	UI section 03 "Files for opentimestamps.org" → Finder *artifacts/ots_public/* → drag the **.ots** then the **.bin** onto opentimestamps.org (Stamp & Verify)	"The same root is also timestamped in Bitcoin. A public website, not ours, checks it: *pending* right after anchoring, then the Bitcoin block and date once confirmed."
9	Section 00 → IMEI **356938035643809** → VERIFIED; Verify stored record → INTACT	"The buyer's view: the server re-computes every hash and asks Sepolia whether the root is anchored."
10	pgAdmin **query 7** (battery cycles 812 → 120) → Verify again	"I'm an insider with full DB access rolling back the cycle count. Verification says **TAMPERED: leaf_digest failed**. I can edit our database, but I can never edit Sepolia."
11	Query 8 (fault code) → report_hash fails; Query 9 restore → INTACT	"Changing the report breaks the report hash; restoring it passes again. The public chain remembers the truth."

Quick fixes if something breaks

- Anything stopped (lid closed, server window closed) → double-click **START_DEMO.command** again.
- Top bar shows *sim · in_memory* → the server could not reach the Sepolia RPC (Wi-Fi / public node down). Check internet, run START again. Without internet START falls back to the local Hardhat chain.
- Audit says *FAIL Etherscan lists 0x…* for the **newest** batch only → Etherscan has not indexed it yet (1–2 min). The tx is already on-chain; rerun the audit a minute later.
- Anchor takes long or errors with funds → Sepolia is congested or the wallet ran out of test ETH; top-up from a Sepolia faucet. Click once and wait; do not double-click.
- Records anchored earlier on the **local Hardhat** chain show *chain_anchor: FAIL* on Sepolia → *python scripts/init_db.py --reset* and seal fresh.
- Top bar shows SQLite → *brew services start postgresql@15*. Port busy → *PORT=8010 bash scripts/start_demo.sh sepolia*.

3b. What is on screen: terminal, Etherscan, web UI and pgAdmin

> "*The database, the AI model and the server run on this laptop. The blockchain is the real public Ethereum Sepolia network: our contract is deployed there, and anyone can check our transactions on Etherscan. Each window is one box of our architecture diagram.*"

Window 1: terminals "Proof of Repair - START" and "Proof of Repair server (Ethereum Sepolia)"

START prints the 5 checks shown in section 3 and exits. The server window keeps running and prints:

Line on screen	What it means / what to say
== 1/4 PostgreSQL	Starts PostgreSQL 15 with the 4,666 repair guides. "Our off-chain registry."
== 2/4 Ethereum (sepolia) contract https://sepolia.etherscan.io/address/0x37C9…	Loads *artifacts/chain_config.sepolia.json* (RPC URL, contract address, ABI). The contract is **already deployed** on Sepolia, so nothing is deployed at start-up.
== 3/4 OpenTimestamps: live	Roots also go to the public OpenTimestamps calendars → Bitcoin.
== 4/4 API + web UI → http://127.0.0.1:8000	FastAPI server is up; */docs* lists every endpoint.
GET /api/health 200 (every 60 s)	The dashboard's health check: the server asks Sepolia for the latest block number and the wallet balance (that is the live block number in the top bar).
POST /api/diagnose_and_prove 200	A repair was diagnosed and sealed (local only, no blockchain).
POST /api/anchor_batch 200	A batch was anchored on Sepolia. This line appears only **after** the block is mined, hence the ~15–30 s pause. The server then also writes the opentimestamps.org file pair to *artifacts/ots_public/*.
GET /api/vin/…, **GET /api/proofs/…/verify**	Buyer lookup and verification (reads from Sepolia, free).

Unlike the local Hardhat node, the public Sepolia network does not print each call on our screen. Etherscan is the public record of what we sent.

Window 2: Etherscan (the public blockchain)

The transaction page opened from the tx hash. Every field and what to say:

Field on Etherscan	What it means / what to say
Transaction Hash	Unique ID of our anchoring. Same value is in the web UI and pgAdmin (*anchor_batches.chain_tx*).
Status: Success	The contract accepted it (it would revert for an empty root, a root already anchored or a reused batch ID).
Block + confirmations	The Sepolia block that includes it. Confirmations = blocks built on top since; the more there are, the harder it is to undo.
Timestamp	Set by the network, not by us: public proof of **when** the root existed. The contract stores exactly this value (*anchoredAt = block.timestamp*), e.g. batch 1: 28-10-2026 11:17:12 IST in block 12,010,314.
From 0x1338…F609	Our workshop wallet. The server signed the transaction with its private key, which never leaves the laptop (stored in *.env*).
To 0x37C9…b6d9 (Contract)	Our ProofOfRepair contract.
Value: 0 ETH	No money is sent; we only store data.
Transaction Fee, Gas Price	Paid in free test ETH; on Sepolia today the gas price is almost zero, so the fee is ~0.0000000000145 ETH. On mainnet this would be a real cost, shared by the whole batch.

Gas Limit & Usage	Real values: **~906,400 gas used of ~1,199,000 limit** (≈76%). The limit is Sepolia's own estimate × 1.3 + 10,000. Sepolia charges far more for storage than our local node did (~208,000), which is why the code now estimates gas for every transaction.
Method / Input Data	Shows **Anchor Merkle Root** (decoded, because our source code is verified). "Decode Input Data" shows our Merkle root, batch ID and metaURI.
Logs tab	The **MerkleRootAnchored** event: root, batch ID, submitter, time. Apps can subscribe to these events.
Contract page → Transactions / Events	History of every batch ever anchored by our system, readable by anyone. One row shows a red **Fail**: the out-of-gas attempt (nothing stored).
Contract page → **Contract ✓** → **Read Contract**	Call *isRootAnchored* / *getAnchor* / *anchors* straight from the browser: independent verification without our server.

Window 3: the web UI (browser)

Part of the page	What to say
Top bar **LIVE · Ethereum Sepolia · id 11155111 · block N**	Live connection to the public chain; the block number increases every ~12 s. The "Smart contract" link opens Etherscan.
Section 01 (Phone / Car / Appliance)	Device ID + fault → manual search → Mistral-7B + LoRA writes JSON → report hash + leaf digest on the hash plate.
Grounded on card	The real iFixit / MechanicDB guide the model used.
Section 03: Anchor pending batch	Merkle root → Sepolia transaction (clickable hash, "Ethereum Sepolia testnet · chainId 11155111 · block … · contract …") → OTS receipt.
Section 00: buyer lookup	Timeline with a tx link per repair + VERIFIED / TAMPERED with the exact failed check.

Window 4: pgAdmin (the database)

- Tables: **assets**, **repair_records**, **anchor_batches** (merkle_root, chain_network = sepolia, chain_tx, block_number, contract_address, **explorer_url**, ots_status), **record_anchors**, **repair_guides**.
- Queries 1–3: knowledge base. 4–6: sealed repairs and their Sepolia anchors. **7–9: tamper demo.**
- "The database is editable; any edit is detected, because the fingerprint is on a public blockchain we do not control."

How the windows connect

```
Browser  --HTTP-->  FastAPI server (Window 1)  --SQL-->  PostgreSQL (pgAdmin)
                          |  --loads-->  Mistral-7B + LoRA adapter (MLX, laptop GPU)
                          |  --JSON-RPC over HTTPS-->  ethereum-sepolia-rpc.publicnode.com
-->  Sepolia network
                          |                                    -> ProofOfRepair contract
0x37C9…b6d9  (seen on Etherscan)
                          |  --HTTPS-->  OpenTimestamps calendars  -> Bitcoin
```

3c. What the server does on Sepolia (which code runs when)

The **ProofOfRepair.sol** contract has three functions we use:

Contract function	Type	What it does

anchorMerkleRoot(root, batchId, metaURI)	Write: a signed **transaction**, costs gas, goes into a block	Stores root + *block.timestamp* + batch ID + submitter; emits *MerkleRootAnchored*. Rejects empty root, root already anchored, reused batch ID.
isRootAnchored(root)	Read (eth_call), free	Is this root on-chain? true/false.
getAnchor(root)	Read (eth_call), free	Returns root, anchoredAt, batchId, metaURI, submitter.

A. Health check (every 60 s)

app.js *refreshHealth()* → **GET /api/health** → *Web3Ledger.status()* → RPC calls **eth_blockNumber** + **eth_getBalance** to Sepolia. That fills the top bar's live block number and lets us see the wallet still has test ETH.

B. "Anchor pending batch": step by step

Browser → **POST /api/anchor_batch** → *anchoring.anchor_batch()* builds the Merkle tree → **Web3Ledger.anchor_merkle_root()** in *evm_ledger.py*:

#	RPC call to Sepolia	Code / meaning
1	eth_call → **isRootAnchored**	Pre-check so the same root is never anchored twice.
2	**eth_estimateGas**	*_gas_limit()*: asks Sepolia how much gas anchorMerkleRoot needs (~915,000); limit = estimate × 1.3 + 10,000 ≈ 1,199,000. Actual use ≈ 906,400.
3	**eth_getTransactionCount** (pending)	Gets the wallet's **nonce** (transaction counter) so every transaction is unique and in order.
4	**eth_maxPriorityFeePerGas**, **eth_getBlockByNumber**(latest)	Sets the fee (EIP-1559): tip + max fee = 2 × current base fee + tip, so it is included quickly.
5	(no network call)	**Signs the transaction locally** with the wallet's private key from *.env*. The key is never sent anywhere.
6	**eth_sendRawTransaction**	Broadcasts the signed transaction to the Sepolia network. **The actual write.** Returns the tx hash.
7	**eth_getTransactionReceipt** (repeated)	*wait_for_transaction_receipt()* (up to 180 s): waits until a validator puts it in a block (~12–30 s), checks status = 1 (success).
8	eth_call → **getAnchor**	Reads the root back from the contract to confirm, takes the on-chain timestamp, builds the **Etherscan link**.

Then the server submits the same root to OpenTimestamps (status **pending**), writes the *.bin* + *.bin.ots* pair to *artifacts/ots_public/*, and saves tx hash, block, contract, Etherscan URL and OTS receipt in PostgreSQL *anchor_batches*.

> *"When I click Anchor, the server checks the root is new, estimates the gas, signs the transaction with the workshop's key on this laptop, broadcasts it to the public Ethereum test network, waits for a block, and reads the root back to confirm. One transaction covers the whole batch, and it is now public and permanent."*

C. Buyer clicks "Verify stored record"

GET /api/proofs/{id}/verify → *verify_full()*: report_hash → leaf_digest → evidence → merkle (all on the laptop), then **chain_anchor** = *ledger.is_root_anchored()* = one free **eth_call isRootAnchored** to Sepolia. No transaction, no fee, no wallet needed.

> "*Verifying is free: it only reads the public chain. A buyer, dealer or insurer could do this check themselves, without trusting our server.*"

In the tamper demo the Sepolia anchor is untouched, so chain_anchor still passes. What fails is the local hash check (leaf_digest / report_hash): the edited record no longer hashes to a leaf inside the anchored root.

Local Hardhat vs Sepolia (if asked "what changed?")

	Local Hardhat (earlier)	Sepolia (now)
Who runs the chain	Our laptop	Public Ethereum test network, thousands of nodes
Signing	Node's unlocked test account (*eth_sendTransaction*)	Our own key, signed locally (*eth_sendRawTransaction*)
Speed	Instant	~12–30 s per anchor (real block time)
Persistence	Lost when the node stops	**Permanent and publicly visible** on Etherscan
Cost	Free	Free test ETH from a faucet; same code would run on mainnet with real ETH

3d. The two timestamps and the audit (what is new)

> "*Every batch gets two independent public timestamps: Ethereum says when our contract stored the root, and Bitcoin, through OpenTimestamps, proves the root existed at that time on a second, unrelated blockchain. And an audit script checks that our database and the blockchain agree.*"

	Ethereum timestamp	Bitcoin timestamp (OpenTimestamps)
Where it comes from	*block.timestamp* of the Sepolia block, saved by the contract as *anchoredAt*	Root sent to public OTS calendar servers; they batch thousands of hashes into one Bitcoin transaction
How fast	~12–30 s (one block)	**pending** immediately, **confirmed** after a few hours (next Bitcoin block that includes the calendar's commitment)
Where to see it	Etherscan Timestamp; UI shows the date (local time); *getAnchor()* returns it	UI "OpenTimestamps receipt · pending/confirmed"; pgAdmin *anchor_batches.ots_status*; opentimestamps.org; mempool.space block link
Who must you trust	The Ethereum network	Only Bitcoin; the calendar cannot fake it
Status on 28-10, 11:40	3 batches, 11:17 / 11:25 / 11:40 IST	all 3 **pending** (anchored this morning)

How a Bitcoin timestamp gets confirmed: *scripts/ots_status.py*

1. Reads each *.ots* receipt in *artifacts/ots_calendar/* (with *--db-only*: only batches in PostgreSQL).
2. For each *pending* attestation, asks that calendar server whether the hash is now in a Bitcoin block, and merges the returned proof into the *.ots* file.
3. **Independent check:** recomputes the proof's final hash and compares it with that Bitcoin block's Merkle root fetched from **mempool.space**.

4. Prints **CONFIRMED … Bitcoin block N** + a mempool.space link, and updates PostgreSQL: *ots_status* = '*confirmed*', *meta.bitcoin_block* = *N*. Otherwise prints **pending … waiting for a Bitcoin block**.

Checking it on opentimestamps.org: *artifacts/ots_public/*

- After every anchor the server (and *scripts/export_ots_files.py*) writes two files per batch, named *<date_time>_<batch>*, e.g. *2026-10-28_1117_sepolia-demo-1793166425.bin* + *.bin.ots*.
- **.bin** = the exact bytes whose SHA-256 is the Merkle root: *0x01 ‖ left child ‖ right child* for a multi-record batch, or *0x00 ‖ leaf payload* for a single record. The script only writes it if it really hashes to the stored root.
- **.bin.ots** = the OpenTimestamps receipt for that root. Run *export_ots_files.py* again after *ots_status.py* to copy in the Bitcoin proof.
- On opentimestamps.org drag the .ots, then the .bin: "Pending" now, later "Bitcoin block N attests existence as of <date>".

Database ↔ blockchain audit: *scripts/check_db_chain.py*

For each…	It checks
Anchor batch	Tx receipt exists and succeeded · sent to our contract · block number matches the DB · the on-chain *MerkleRootAnchored* event carries the same root and batch ID · *isRootAnchored(root) == true*
Repair record	Report hash recomputes · leaf digest recomputes · stored Merkle path ends at the batch root · path proves the leaf is in the root
Etherscan	Every DB transaction is listed on Etherscan, and every successful anchor on Etherscan is in the DB (reverted ones are reported as a note)

Ends with **RESULT: DATABASE AND BLOCKCHAIN ARE CONSISTENT** (exit code 0) or **MISMATCH FOUND**. Ran at 11:42: all record and chain checks **OK** for 3 batches / 5 records; only the newest tx (anchored 11:40) was not yet listed by Etherscan's indexer, so rerun it a minute after anchoring. After the tamper query (7 or 8), this audit also flags that record.

4. Limitations (say them before they ask)

- **Integrity, not quality:** verification proves the record was *not changed after sealing*. It does not prove the repair was done well or that the input was truthful ("garbage in, sealed garbage out"). That is why workshop review + signatures are future work.
- **No physical OBD-II integration:** codes are typed in manually; demo evidence is synthetic.
- **Testnet, not mainnet:** anchoring runs on the public Sepolia *test* network (test ETH has no value). Mainnet would need real ETH or a cheaper Layer-2. Anchoring depends on a public RPC node and takes ~12–30 s. One wallet key in *.env* signs for all workshops.
- **Small test sets:** 57 OBD cases and 30 multi-domain cases; results are indicative.
- **Taxonomy:** car codes are real OBD-II; phone/appliance codes (PH-BATT, AP-WATER …) are defined by us. The code is assigned deterministically (OBD code, else issue text + matched manual); the LLM writes the diagnosis, steps, parts, QA checks and advice.
- **Training text:** symptom wording for iFixit examples is templated per component; procedures, parts and tools are real.
- **No authentication yet:** any workshop could seal; no digital signatures. Not production-deployed in the cloud.
- **Speed:** ~13 s per diagnosis for Mistral-7B on a laptop.

5. Likely questions, short answers

Q	A
Why not store the whole record on the blockchain?	Cost and privacy. One 32-byte root covers a whole batch; personal data stays off-chain.
Can the admin edit the database?	Yes, but it is detected: recomputed hashes stop matching the anchored root (shown in demo).
Can the admin delete a record?	A deleted record cannot be verified any more, and the anchored root shows a batch existed. Detecting a hidden deletion needs a full batch manifest/signed index; future work.
Why is there a failed transaction on Etherscan?	Our first Sepolia anchor used the old fixed 500,000 gas limit; Sepolia needed ~906,000, so it ran out of gas and reverted (nothing stored, root not anchored). We fixed it by estimating gas per transaction. The audit reports it as a note.
Why ~906k gas on Sepolia but ~208k locally?	Same contract and call; the public network's gas rules price the storage writes much higher than our local Hardhat node. The fee is still almost zero on the testnet.
How do I know the DB and chain agree?	*check_db_chain.py* checks every batch against the chain and Etherscan and every record's hashes: CONSISTENT.
Why is Bitcoin still pending?	OTS calendars aggregate many hashes into one Bitcoin transaction; it is confirmed after the next Bitcoin blocks, usually a few hours. *ots_status.py* then upgrades the receipt and checks it against the block header.
Why Merkle tree?	One transaction for many records; each record proves membership with $\log_2(n)$ hashes.
Why Sepolia and not Ethereum mainnet?	Same code and contract, zero cost for a prototype. Moving to mainnet or a Layer-2 is only a config change (RPC URL, chain id, contract address).
Is it really public?	Yes: anyone can open our contract or tx on sepolia.etherscan.io without our server.
Why two anchors (Ethereum + OTS)?	Independence: if one is unavailable, the other still proves the time of existence. OTS is free.
Why LoRA?	Trains <1% of parameters; a 7B model fine-tunes on a laptop with no extra inference latency.
Why include model provenance in the leaf?	So later you know which model/version wrote the report; auditability if a model is found faulty.
Why TinyLlama 100% JSON but lower accuracy?	Small models trained on a fixed format copy the format well but have less knowledge to pick the right code.
What if the AI is wrong?	Invalid JSON is rejected before sealing; human review before sealing is planned. The seal proves integrity, not correctness.
Verification time?	Under 35 ms for the cryptographic checks (hash, leaf, Merkle path).

Sources: slides 9–13, DEMO.md, START_DEMO.command, scripts/ots_status.py, scripts/export_ots_files.py, scripts/check_db_chain.py, live Sepolia data (28-10-2026 11:42), artifacts/chain_config.sepolia.json, src/blockchain/evm_ledger.py, chain/contracts/ProofOfRepair.sol, artifacts/eval_metrics_multidomain.json, artifacts/dataset_manifest_multidomain.json, src/por/verify.py.